

R0 Review - Walking Skeleton

TalentSignal Revenue Engine · S-01 to S-03 · Revision sheet for Megan

How to use this. Read it once end to end, then take the self-test quiz. Anything you can't explain out loud is a soft spot. The point isn't recall — it's being able to defend every claim on a whiteboard with no notes.

The framing line for the whole demo:

“The AI wrote most of this code. My job was to decide whether it was right — and three times it told me a story was done when it wasn't.”

What R0 is

A walking skeleton: the thinnest possible slice through every layer of the system, with no real features in it. It builds none of the intelligence — it builds the spine and proves the spine works. One command starts a React front end, a Node/Express API and a PostgreSQL database, all networked; one mocked market signal flows through to a scored opportunity on screen.

Why first? Build features on unproven plumbing and every bug becomes four questions at once: is it my feature, the network, Docker, or the database? Prove the skeleton first and a later bug has one fewer place to hide.

The three stories

Story	What it does	Trust scenario
S-01 Skeleton	docker compose up starts all three services; /api/health is green; every request logs JSON with a correlation id.	Failures are observable from commit one, never silent.
S-02 Auth	Register/login; bcrypt-hashed passwords; JWT carrying user id and role; rate-limited auth routes.	Only a bcrypt hash is stored; the raw password never reaches the logs.
S-03 Slice	A mocked signal flows through a provider adapter, gets scored, upserts to Postgres, renders in React.	Confidence and source always shown together — never a bare number.

The eight defects review caught

This is the centre of the demo. Know at least the first three in full detail.

1 · Cold-start race (S-01)

All three containers started at once. Postgres takes seconds to accept connections, so the API came up, tried to connect, and failed. **depends_on waits for a container to START, not to be READY.** Fixed with a pg_isready healthcheck plus depends_on: condition: service_healthy. The acceptance criterion said 'given a fresh clone' — it would have failed on exactly the scenario it was written for.

2 · Privilege escalation (S-02) - most serious

Registration accepted a role from the request body. Anyone could POST role:'admin' and make themselves an administrator — S-14's access control would have been decorative. Fixed: registration hardcodes 'sales'; admin is bootstrapped from environment variables at boot.

3 · Silent logger (S-01)

The logger listened only for the 'finish' event, which fires on normal completion. Close the tab mid-request and finish never fires — **no log line at all.** The defect was inside the safety feature itself. Fixed by also listening for 'close' and using writableEnded to tell completed from aborted.

4 · Flaky migrate test (S-01)

A test asserted the database started un-migrated, making it depend on outside state — a coin flip in CI. Isolated in a throwaway schema. (I suggested just deleting the assertion; Claude Code pushed back with a better reason and isolated it instead.)

5 · No tests in the plan (S-01)

The definition of done requires happy-path, failure and idempotency coverage. The story could not have passed its own bar. Tests added.

6 · 404 on the documented path (S-03)

Routes were served at root; the spec says /api/*. They only worked in a browser because the dev proxy stripped /api. Curling the documented path 404'd — and it would have broken entirely at deployment. Now mounted under /api.

7 · Dead security middleware (S-03)

requireAuth was written in S-02 and applied to nothing. The new endpoints exposing company data were open. Now enforced, with the 401 path tested.

8 · Missing login screens (S-03)

S-02's build note called for register/login screens; it was marked complete without them (which is why a token had to be hand-placed in browser storage). Built now, and corrected publicly on Basecamp rather than folded in quietly.

Concepts you must explain out loud

Structured logging + correlation id

Every log line is a JSON object with the same fields and a unique request id, so one request is traceable through everything it touched. Python analogue: logging with a JSON formatter and a request id in the extra dict.

Why duration is measured on response finish

The handler is async, so measuring right after handing off would read near-zero — long before the response is actually sent. So a callback runs when the response ends. Defined now, runs later.

Why bcrypt is deliberately slow

Slowness is the security. Cost 12 is about a quarter-second per hash, which makes brute-forcing a stolen database expensive. The cost factor is exponential. This conflicts with the p95 target on purpose — escalated to Ali, not silently weakened.

JWT is signed, not encrypted

Anyone can read the payload; only the server, with the secret, can produce a valid signature. It proves not-tampered, not secret. Never put a real secret in the payload.

User enumeration + the timing attack

Wrong password and unknown email return byte-identical 401s (no enumeration), and both run a bcrypt compare — against a dummy hash when there's no user — so neither path is measurably faster (no timing leak). Claude Code caught the timing half unprompted.

The duplicate-email race

Naive check-then-insert lets two simultaneous registrations both pass the check. Only the database can arbitrate: a UNIQUE constraint means one insert wins, the other fails with code 23505, which becomes a 409. Verified live: exactly one 201, one 409, one row.

The adapter seam

The route receives the provider as an argument, not an import. Swapping the mock for a real job board is one line in the startup file; the route, scoring and tests don't change. This is rule 5: transparent heuristic now, trained model later, callers untouched.

Upsert

INSERT ... ON CONFLICT ... DO UPDATE. Re-analyzing the same signal updates the score in place instead of duplicating. Rejecting the second call would hide a real change.

The confidence score - say the honest thing

These weights are **Megan's proposal, pending Ali's approval**. The spec says not to invent them; she asked, got no answer before the deadline, and shipped a documented proposal with every constant in one config object — so his answer is a one-line change.

Factor	Weight	Why a salesperson would accept it
--------	--------	-----------------------------------

Base	0.20	A real posting existing is itself weak evidence
Reposted role	0.32	They tried once and failed to fill it
Days open (max 30)	0.32	The longer it sits, the more they need help
No salary range	0.16	Correlates with less structured hiring

Why the base score exists: without a floor, a job posted yesterday scores 0.0 — zero confidence that a company actively hiring might need help. That's indefensible, so a floor was added. Volunteer this; it shows you interrogated the model.

Caveats to volunteer - before you're asked

- The job board is **mocked**, behind an interface — one line to swap in a real feed.
- The confidence weights are **mine and unapproved**, flagged pending the client.
- Roles are **stored but not enforced** — that's S-14.
- **Nothing is deployed** — it has never faced the internet. That's S-20.
- **Performance is unmeasured** — never claim a number. That's S-18.

“I would rather say I haven't measured that than guess in front of you.”

Questions you will get

Q — Did you write this, or did the AI?

The AI wrote most of the code. I specified, reviewed and approved it — and rejected plenty. Three times it reported a story done when it wasn't. Catching that is the job.

Q — Why does the UI look so plain?

Deliberately. It's a walking skeleton. Styling it would mean building something before proving the layers underneath work.

Q — Where did the confidence weights come from?

Me. A documented proposal, marked pending approval — not client-approved numbers. The spec says these are a business decision I shouldn't invent, so I asked, got no answer before the deadline, and shipped a transparent proposal rather than miss it or hide the guess.

Q — So a job posted yesterday is 20% likely to be an opportunity?

Not a probability — a relative confidence score. 0.20 is the floor, meaning a real posting is weak evidence on its own. Without a floor a brand-new posting scores zero, which would say a company actively hiring definitely doesn't need help. That's wrong.

Q — How do you know the password isn't logged?

A test registers with a distinctive password and asserts the string appears nowhere in the captured logs — full-text search, so it catches leaks through error messages and stack traces too.

Q — Is it secure?

Authentication yes — hashed passwords, expiring tokens, rate limiting, no user enumeration. Authorization no, not yet: roles are stored but nothing enforces them (S-14). And nothing's deployed, so it hasn't faced the internet.

Q — Can it handle 1000 concurrent users?

No idea. That's S-18 and I haven't measured it. I'd rather say that than guess.

Q — Why one-hour tokens with no refresh?

A deliberate tradeoff — refresh machinery deserves building once, properly. The cost is a stolen token stays valid up to an hour. Logged as decision 002. I originally argued 15 minutes plus refresh; Claude Code made the better argument and I changed my mind.

R0 proved the spine. R1 makes it intelligent: hidden-demand analysis, clients and candidates with PII tagged, matchmaking, and idempotent opportunity scoring.